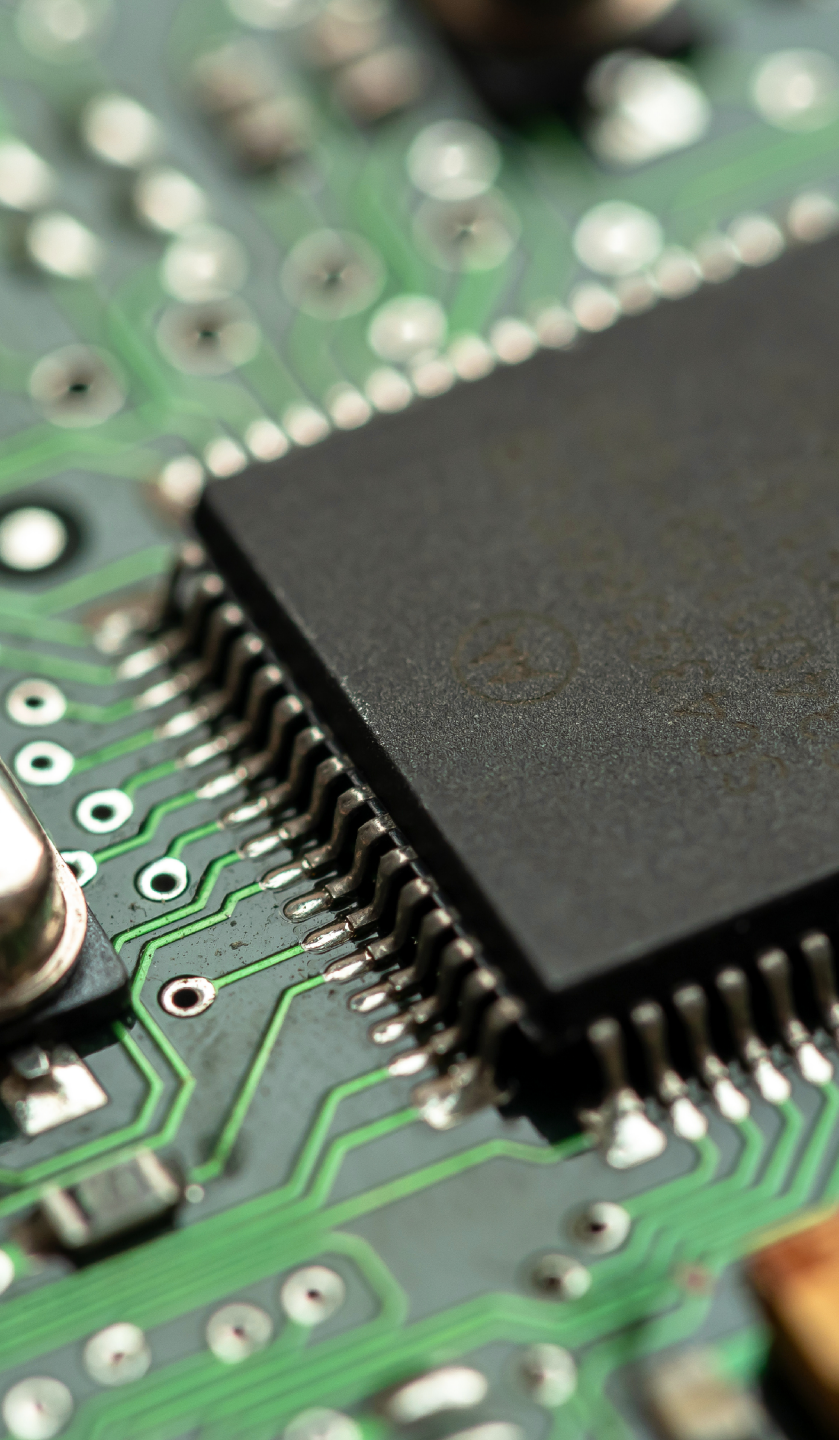




II2001 Mikroprozessortechnik

Lerneinheit 3: Bestandteile eines Mikroprozessors und digitale Ein-/Ausgabe

Ing. Jakob Czekansky, M.Sc.

Sommersemester 2026

Technische Hochschule Mittelhessen

Ingenieur-Informatik (B. Sc.)

Informatik (B. Sc.)

Inhalt

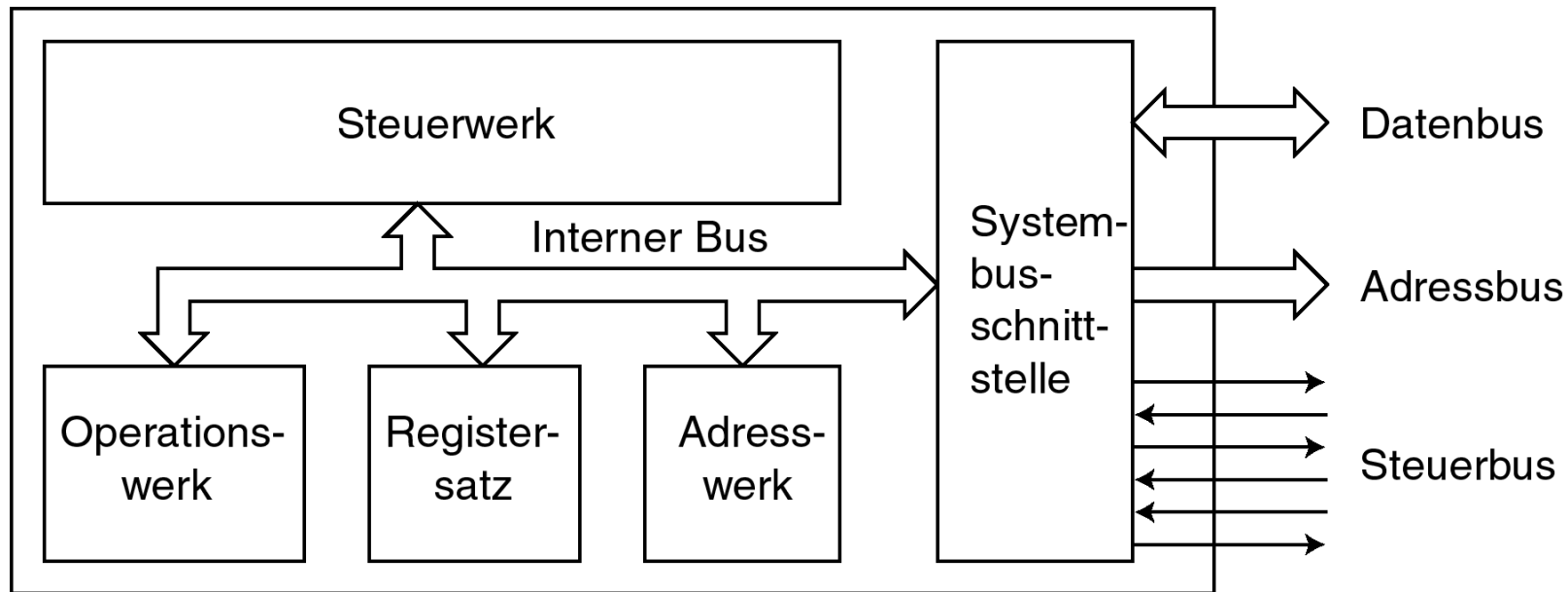
- Die Bestandteile eines Mikroprozessors
 - Befehle und Daten
 - Busleitungen
 - Busarten
 - Verbindungen zur Außenwelt: Ein- und Ausgabe
 - Von der elektrischen Spannung zum Bit
 - Störabstand

Inhalt

- Kurzeinführung: AVR128DB48
- Digitale Ein-/Ausgabe
 - Allgemeines
 - Digitale Schaltung einer Ein-/Ausgabe Leitung
 - Digitale Ein-/Ausgabe bei Mikrocontrollern
 - Fallbeispiel: IO des AVR128DB48 ("Pinout")
 - Auslesen von Schaltern und Tastern
 - Pull-Up und Pull-Down Widerstände
 - Prellen

Interner Aufbau eines Mikroprozessors

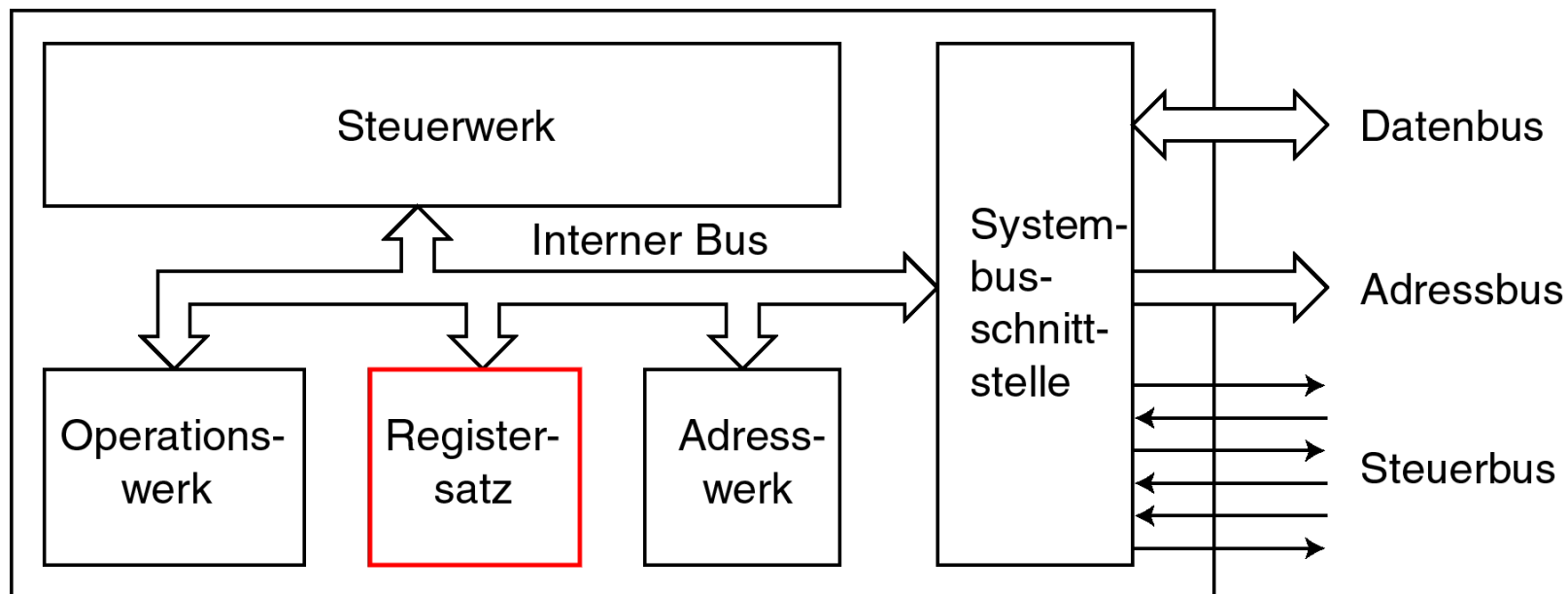
Alle Mikroprozessoren bestehen in ihrem Inneren aus mehreren Baugruppen, die für verschiedene Aufgaben zuständig sind.



Blockschema: Interner Aufbau eines Mikroprozessors

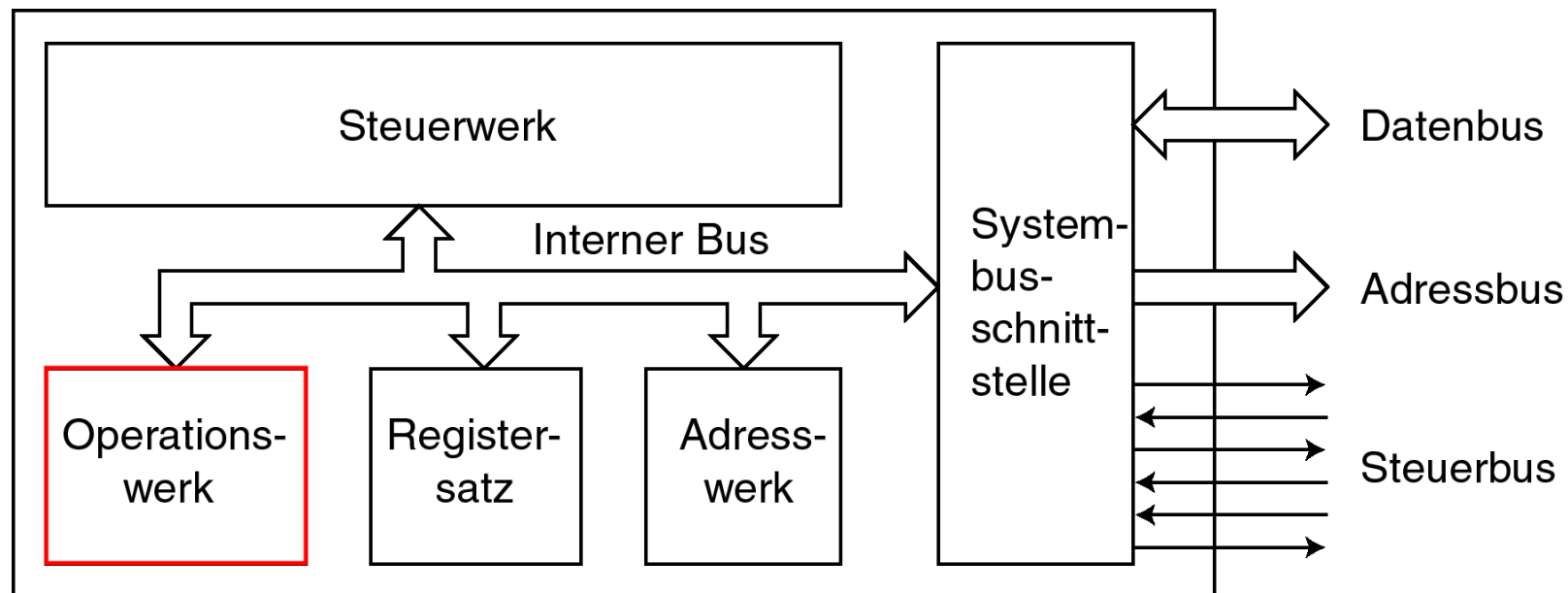
Interner Aufbau eines Mikroprozessors - Registersatz

Der Registersatz ermöglicht das Speichern von Daten innerhalb des Prozessors. Ein Register ist eine steuerbare Gruppe von Flipflops.



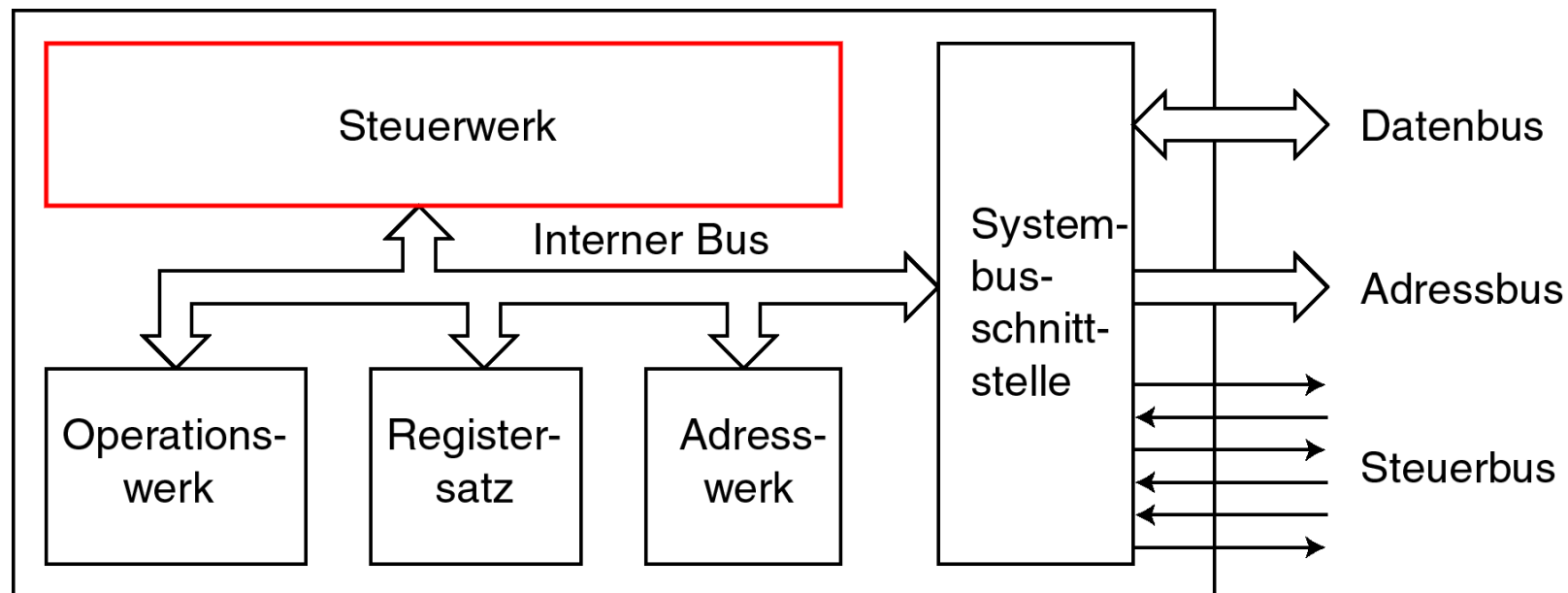
Interner Aufbau eines Mikroprozessors - Operationswerk

Das Operationswerk führt die eigentliche Verarbeitung, d.h. die logischen und arithmetischen Operationen an den übergebenen Daten aus.



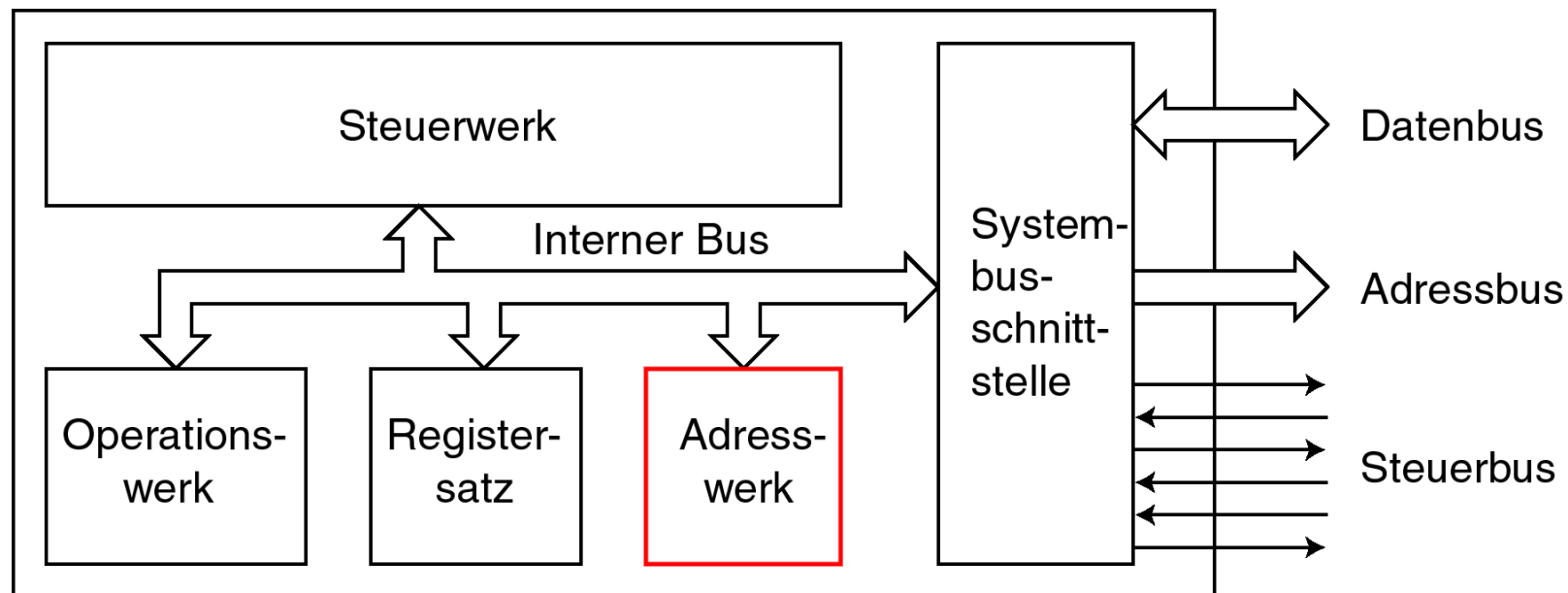
Interner Aufbau eines Mikroprozessors - Steuerwerk

Das Steuerwerk ist verantwortlich für die Ablaufsteuerung sowohl im Inneren des Prozessors als auch im restlichen System.



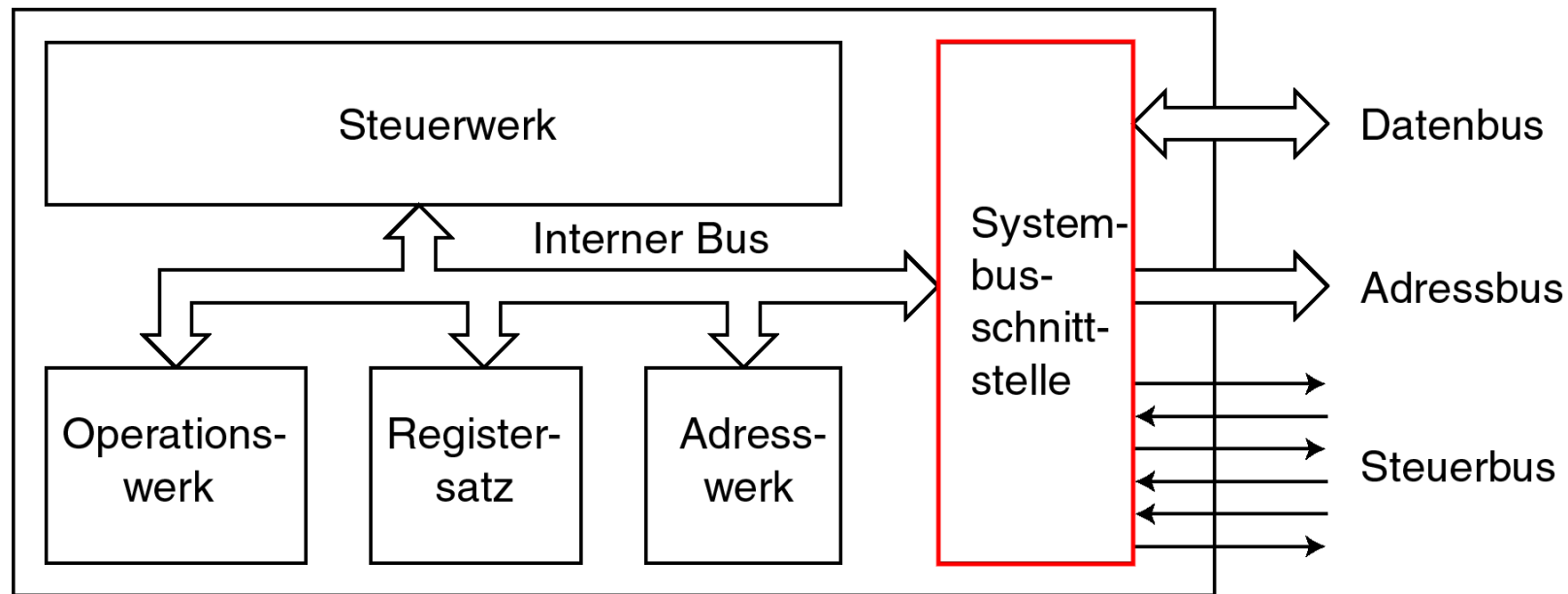
Interner Aufbau eines Mikroprozessors - Adresswerk

Das Adresswerk erzeugt die erforderlichen Adressen, um auf Daten und Code im Hauptspeicher zugreifen zu können.



Interner Aufbau eines Mikroprozessors - Systembus

Die Systembus-Schnittstelle enthält Puffer- und Treiberschaltungen, um den Datenverkehr über den Systembus abzuwickeln.



Zusammenfassung der Vorherigen Folien

- Der **Registersatz** ermöglicht das Speichern von Daten innerhalb des Prozessors. Ein Register ist eine steuerbare Gruppe von Flipflops.
- Das **Operationswerk** führt die eigentliche Verarbeitung, d.h. die logischen und arithmetischen Operationen an den übergebenen Daten aus.
- Das **Steuerwerk** ist verantwortlich für die Ablaufsteuerung sowohl im Inneren des Prozessors als auch im restlichen System.
- Das **Adresswerk** erzeugt die erforderlichen Adressen, um auf Daten und Code im Hauptspeicher zugreifen zu können.
- Die **Systembus-Schnittstelle** enthält Puffer- und Treiberschaltungen, um den Datenverkehr über den Systembus abzuwickeln.

Befehle und Daten

Oft spricht man auf dieser Ebene der Computersteuerung von **Befehlen** (auch Instruktionen, Anweisungen, o.Ä.) und **Daten** (sg. Datum).

- Befehle sind binär kodierte Bitmuster, die den Prozessor anweisen, bestimmte Operationen auszuführen.
- Daten sind binär kodierte Bitmuster, bestimmter Länge (z.B. 8, 10, 12, 16 oder 32-Bit). Sie werden je nach Befehl auf bestimmte Art interpretiert, z.B. als vorzeichenlose 32-Bit-Zahl oder als 8-Bit ASCII Zeichen.

Busleitungen

Busleitungen oder einfach **Busse** sind Leitungsbündel, die parallel angeschlossen sind.

Bei einer Übertragung transportiert jede Busleitung ein Bit, wobei die Einsen und Nullen der Bitmuster auf den Busleitungen als HIGH- und LOW-Pegel dargestellt werden.

D7	LOW
D6	LOW
D5	LOW
D4	HIGH
D3	HIGH
D2	LOW
D1	HIGH
D0	HIGH

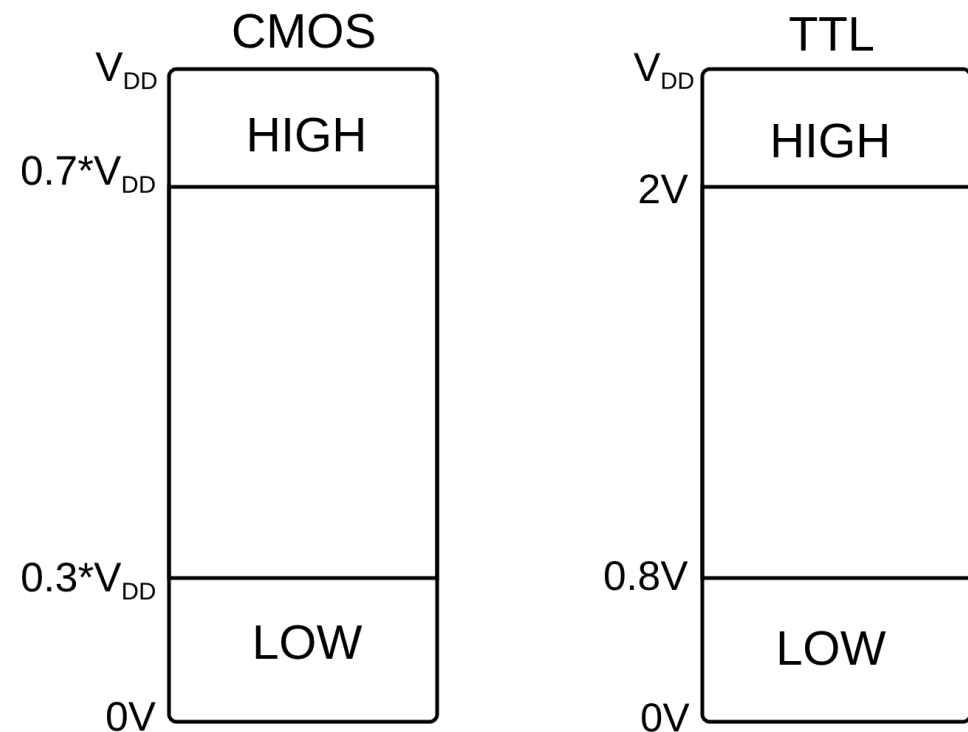
Die Übertragung des Bitmusters 0b00011011 auf einem 8-Bit-Datenbus

Die Unterscheidung zwischen HIGH und LOW

CMOS und TTL sind Standards zur Verarbeitung von Integrierten Schaltungen.

Sie definieren u.A. den Bereich zwischen HIGH und LOW, um resistent gegen sog. **Störungen** (ungewollte elektrische Einflüsse) zu sein.

Dieser Bereich wird **Störabstand** (engl: noise margin) genannt.



Anmerkung: V_{DD} ist bei CMOS variabel, bei TTL konstant 5V (außer LVTTTL)

Busarten

- **Datenbus** Leitungen, über die **Befehle** und **Daten** übertragen werden.
- **Adressbus** Leitungen, mit denen in Speicherbausteinen und Ein-/Ausgabebausteinen bestimmte Speicherplätze angewählt werden. Die übermittelten Bitmuster sind die Adressen. Mit n Adressleitungen können 2^n Adressen angesprochen werden.
- **Steuerbus** Leitungen, mit denen Bausteine in Ihrer Arbeitsweise gesteuert und koordiniert werden. Besteht aus vielen unterschiedlichen Steuerleitungen. Im Gegensatz zum Datenbus und Adressbus hat im Steuerbus jede Leitung eine ganz spezielle Bedeutung.

Ein- und Ausgabe

Mikrocontrollersysteme sind meist durch sogenannte **Peripherien** mit der Außenwelt verbunden. Diese sind **Sensoren** und **Aktoren**, oder andere (Mikro-) Computer.

Der Mikrocontroller selbst verwendet Eingabebausteine und Ausgabebausteine, um mit solchen Peripherien zu kommunizieren.

Eingabebausteine

Nehmen Signale von Peripherien **entgegen**. Der Eingabebaustein übermittelt diese dem Prozessor über den Datenbus.

Ausgabebausteine

Geben Signale an externe Peripherien **aus**.
Der Prozessor übermittelt Bitmuster via den Datenbus an die Peripherie.

Erdung

Alle Peripherien benötigen eine gemeinsame Erdung (Ground), um zu kommunizieren. Ist dies nicht gegeben, können Spannungen (und damit logische Werte) **nicht korrekt interpretiert werden**. Wenn eine gemeinsame Erdung gegeben ist, müssen den Spannungen nun Werte zugewiesen werden! Bei positiver Logik ist ein hoher Pegel (HIGH) ein 1 Bit.

Gibt es keine gemeinsame Erdung, können Signale **fehlinterpretiert** werden oder im schlimmsten Fall **Geräteschäden** entstehen!

Digitale Ein- und Ausgabe

Allgemeine Funktionsweise

Die digitale Ein- und Ausgabe ist die klassische Methode, um Signale bzw. Daten zwischen digitalen Systemen auszutauschen. Auf den Leitungen gibt es nur HIGH/LOW.

Andere Bezeichnungen:

- Eingabe/Ausgabe, E/A
- Input/Output, I/O, IO, Digital-IO
- I/O-Ports (port = Hafen, Anschluss) oder einfach Ports

Eingabe und Ausgabe

Ausgabe:

- Ein Bit im Programm wird in einen Pegel auf der Portleitung umgesetzt
- Aus 0 wird **LOW**
- Aus 1 wird **HIGH**

Durch Ausgabe kann ein elektrisches Gerät hängen und damit **gesteuert** werden. (z. B. eine LED oder ein Motor)

Eingabe:

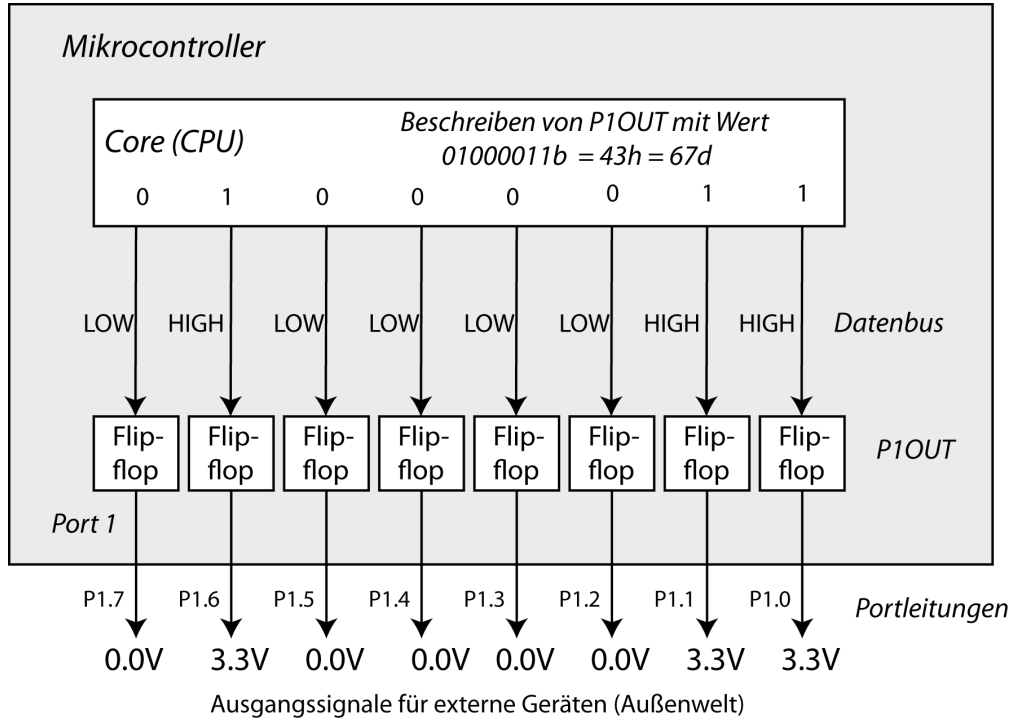
- Ein Pegel auf einer Portleitung wird in ein Bit des Programms umgesetzt
- Aus **LOW** wird 0
- Aus **HIGH** wird 1

Auf diese Art kann das Programm Informationen erhalten, die der Mikrocontroller **nicht selbst erfassen kann**. (z.B. Sensor, Schalter)

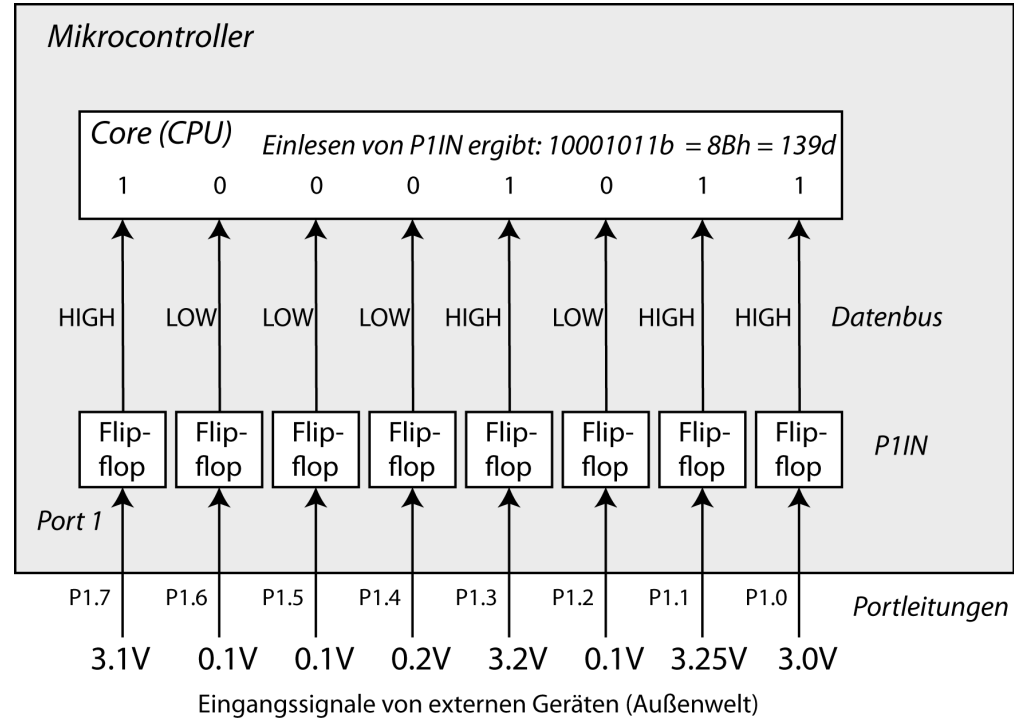
Es gibt Peripherien, die jeweils nur Aus-/Eingabe beherrschen, oder beides. Auch gibt es sog. **negative Logik**, die die Logik vertauscht (0 = HIGH, 1 = LOW)

Ausgabe und Eingabe grafisch

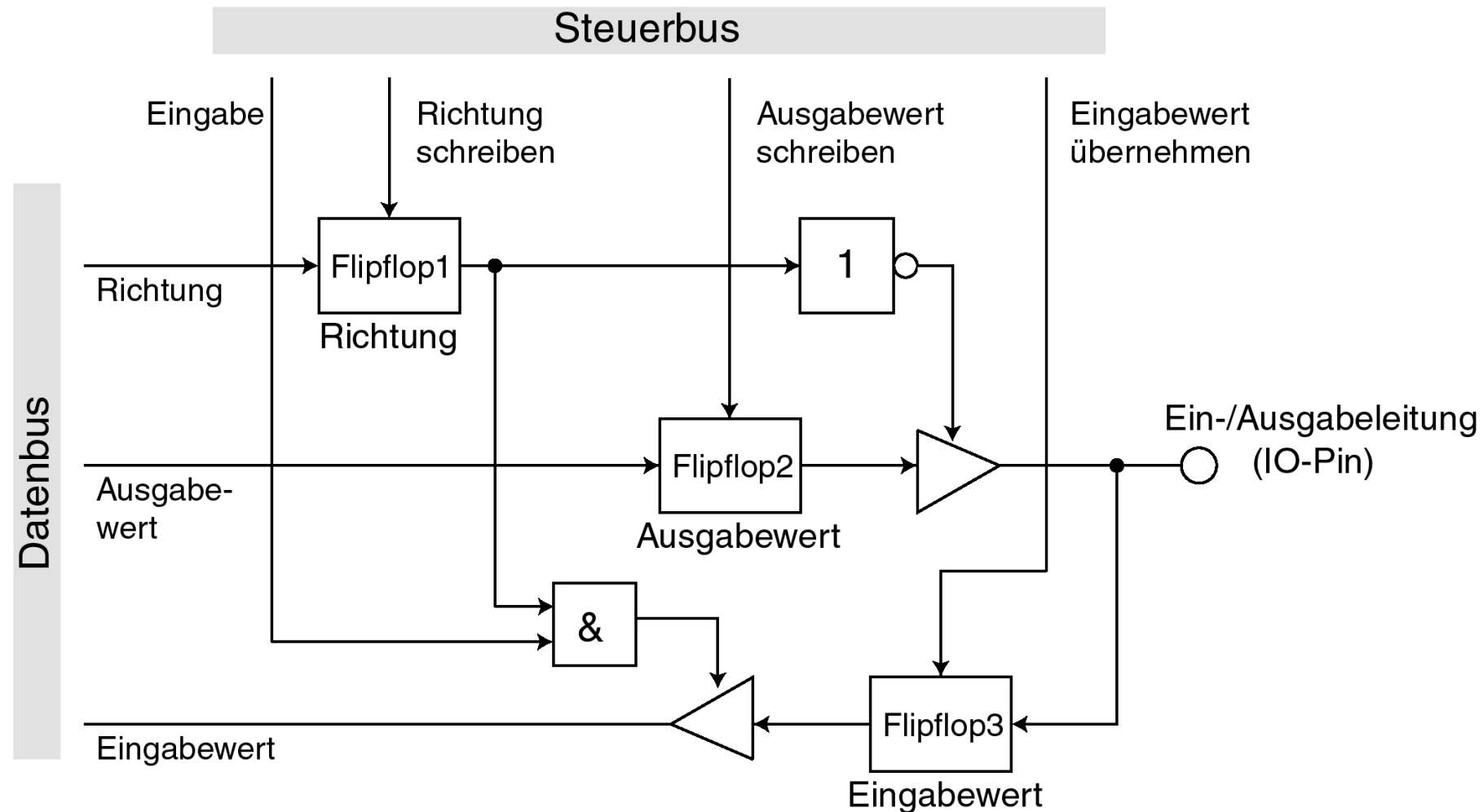
Ausgabe



Eingabe



Digitale Schaltung einer Ein-/Ausgabe Leitung



Digitale Schaltung einer Ein-/Ausgabe Leitung

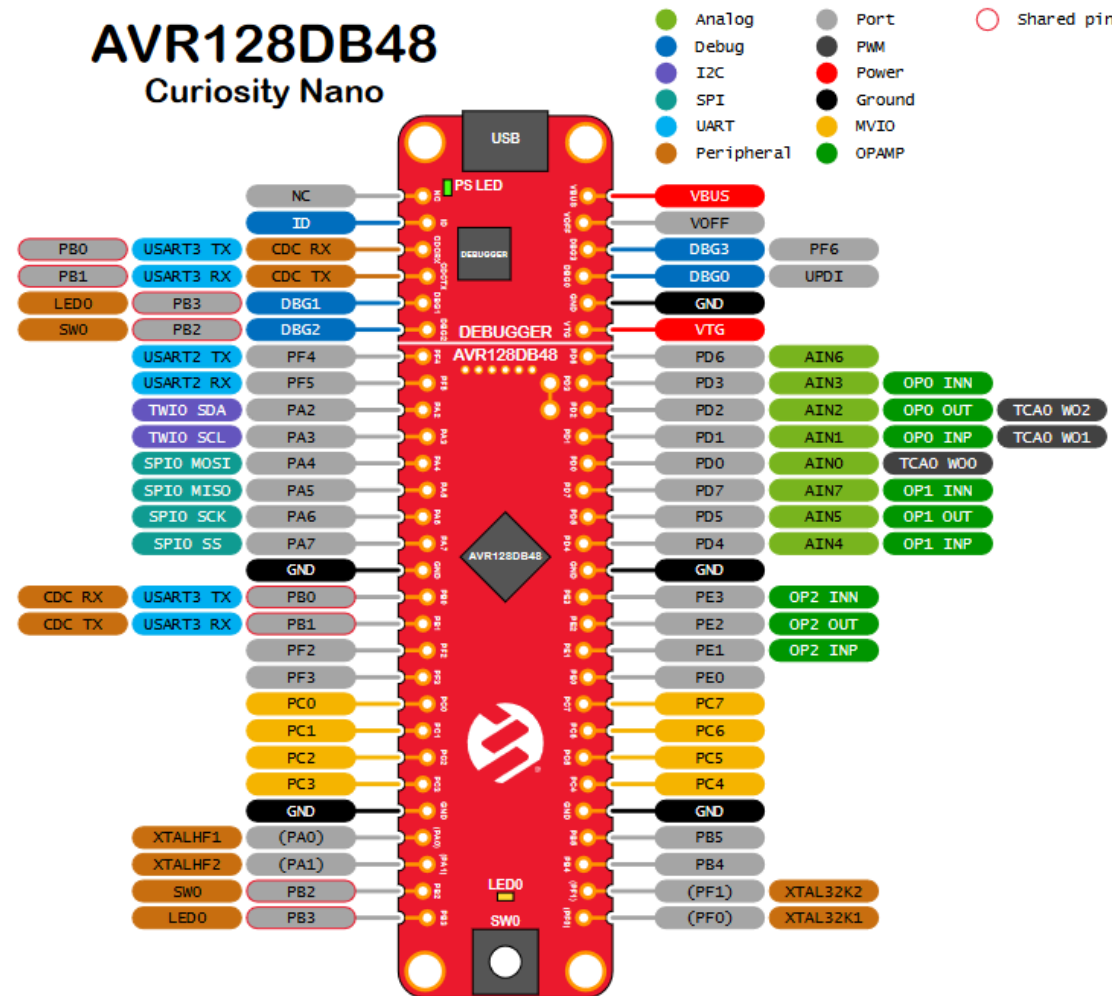
Eine etwas flexiblere Schaltung, die wahlweise für Eingabe oder Ausgabe benutzt wird.

- die Richtung wird in **FlipFlop1** bestimmt: **1=Eingabe** , **0=Ausgabe** .
- Ist die Richtung **"Eingabe"** gewählt, kann durch das Signal "Eingabewert übernehmen" das an der Ein-/Ausgabeleitung anliegende Signal im **Flipflop3** eingespeichert werden.
- Durch das Steuersignal "Eingabe" wird es auf den Datenbus eingekoppelt.
- Ist dagegen die Richtung "Ausgabe" gewählt, so wird der in **Flipflop2** gespeicherte Wert in ein TTL-Signal gewandelt und dauerhaft auf die Ein-/Ausgabeleitung gegeben.
- Mit dem Steuersignal "Ausgabewert schreiben" kann in **Flipflop2** ein neuer Wert eingetragen werden.

Digitale Ein-/Ausgabe bei Mikrocontrollern

- Für Mikrocontroller besonders wichtig: Austausch **digitaler Signale** mit dem umgebenden System.
Beispiele: Ansteuern LED (Ausgabe), Einlesen Schalter (Eingabe)
- Alle Mikrocontroller haben mehrere IO-Ports, welche meistens in Gruppen zu **8 Bit** organisiert sind.
- Port hat meistens **Datenregister** und **Richtungsregister** (Eingabe oder Ausgabe?)
- Oft rücklesbar, im Ausgabebetrieb kann Wert vom Controllerkern wieder eingelesen werden.
- Schaltungstechnik: Ausgangsstufen sind **Open-Collector** -, **Open-Drain** - oder **Gegentakt-Endstufen** .

Fallbeispiel: IO des AVR128DB48 ("Pinout")



Beispiel: Funktionen von Pin PC5

Pin PC5 kann vier verschiedene Funktionen annehmen:

1. Die Leitung PC5 vom GPIO-Port C als digitalen Ein-/Ausgabepin bereitstellen,
2. USART Nachrichten empfangen (USART1 RxD),
3. SPI Nachrichten von Peripherien empfangen (SPI1 CIPO),
4. als Ausgang für hardwaregesteuerte PWM Signale verwendet werden.

Pin Funktionen

Die Auswahl der Funktion eines Pins findet über das jeweilige **PORTMUX Register**, sowie die jeweiligen Peripherieregister.

Ist eine Funktion ausgewählt, werden die Konfigurationen der Alternativregister ignoriert.

- Die digitalen Ein-/Ausgänge (I/O-Ports, oder einfach Ports) sind mit **PA**, **PB**, **PC** ... bezeichnet.
- Jede **Leitung** kann unabhängig als Ein- oder Ausgang eingestellt werden
- Zu jedem IO-Port gehören **mehrere Konfigurations-Register**
- Die Konfigurations-Register der IO-Ports haben 8 Bit
- Die 8 Bit sind den 8 Leitungen **zugeordnet**, also Bit 0 gehört jeweils zu Leitung 0, Bit 1 zu Leitung 1

Beispiel: die Register für Port C:

PORTC.DIR – Port C Data Direction Register (Richtungsregister)

- Bit = 0: zugehörige Leitung ist Eingangsleitung
- Bit = 1: zugehörige Leitung ist Ausgangsleitung

PORTC.OUT – Port C Output Value Register (Ausgangspufferregister)

- Bit = 0: zugehörige Ausgangsleitung wird auf LOW geschaltet
- Bit = 1: zugehörige Ausgangsleitung wird auf HIGH geschaltet

PORTC.IN – Port C Input Value Register (Eingangspufferregister)

- Wenn die zugehörige Eingangsleitung LOW ist: Bit = 0
- Wenn die zugehörige Eingangsleitung HIGH ist: Bit = 1

Auslesen von Schaltern und Tastern

Schalter und Taster sind das **MMI** (Man-Machine-Interface) für die Eingabe, damit lassen sich Mikrocontroller von Bedienern steuern.

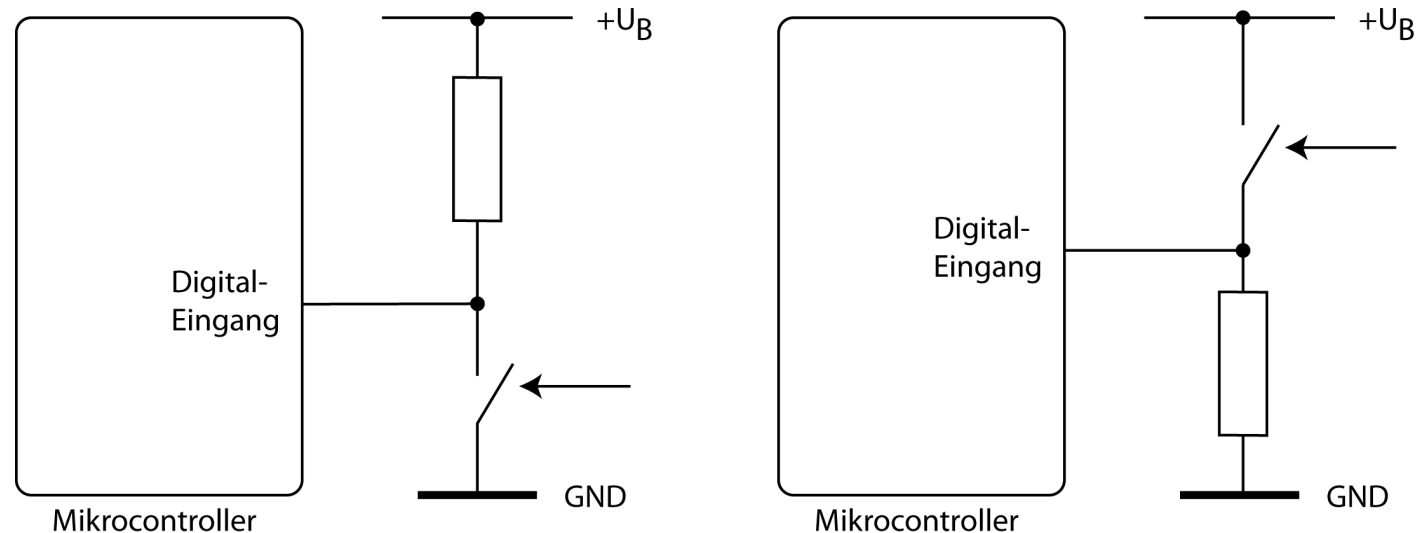
Sie werden meistens an **Digitaleingänge** angeschlossen.

Beim Auslesen von Tastern (Drück-Schalter mit nur einer stabilen Stellung) muss man zunächst herausfinden, ob sie in der Ruhestellung **HIGH oder LOW** liefern.

Pull-Up und Pull-Down Widerstände

Wenn ein Taster nicht gedrückt ist, ist seine Leitung offen. ("floating"), was zu unvorhersehbarem Systemverhalten führen kann. Um das zu verhindern, verwendet man einen **hohen Widerstand** der das Signal definiert.

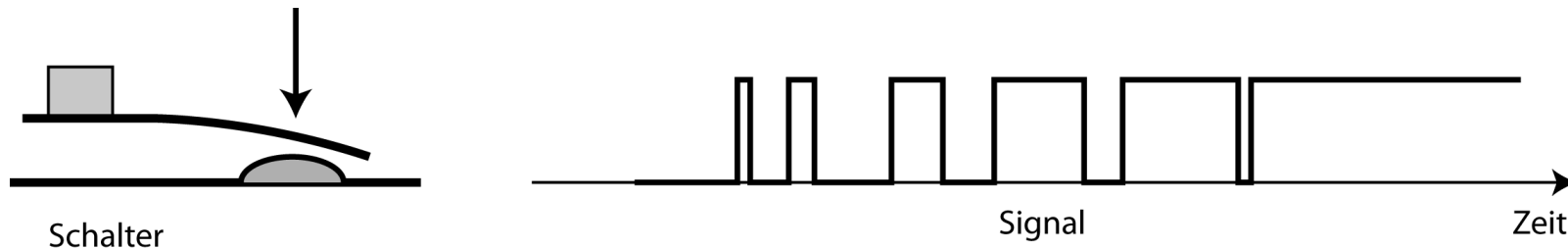
Verbindet der Schalter die Leitung zu Ground (LOW), braucht man einen **Pull-Up** Widerstand (Widerstand zu HIGH), Gegenstück: **Pull-Down**.



Prellen

Wenn der Taster gedrückt wird, schließt er nicht sauber in einem Schritt, sondern schließt mehrfach kurz, bis er endgültig Verbindung hat.

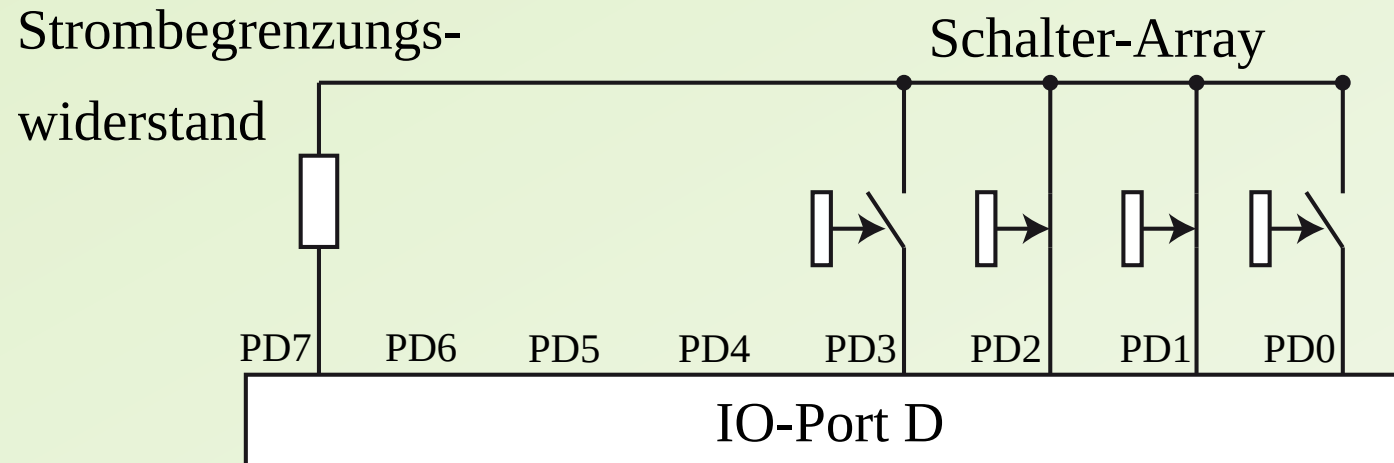
Praktisch alle Schalter prellen, sie kontaktieren also **mehrfach**, bis der sichere Kontakt (Schluss) besteht.



Taster und Schalter kann man softwareseitig oder hardwareseitig entprellen. Mögliche Lösungsansätze sind **Delays** (blockierend/nicht-blockierend), **Zustandsautomaten**, **Tiefpassfilter** (RC-Glied), etc.

Übung: IO Konfiguration Eingabe

Betrachten Sie die unten dargestellte IO-Schaltung:



1. Wie muss Port D konfiguriert werden, um die Schalter auszulesen?
2. Welches Bitmuster (= Zahlenwert) wird man im Datenregister von Port D lesen, wenn zwei der Schalter gedrückt sind, wie im Bild dargestellt?
3. Schreiben Sie den vollständigen C-Code auf, um den Schalter auszulesen und das Ergebnis auf die Variable **switch1** zu übertragen!

Übung: IO Konfiguration Ausgabe

Betrachten Sie die unten dargestellte IO-Schaltung:

- Wie muss Port D konfiguriert werden, um das LED-Array anzusteuern?
- Welche Bitmuster (=Zahlenwerte) müssen nacheinander auf Port D geschrieben werden, um ein im Uhrzeigersinn umlaufendes Licht zu erzeugen?
- Schreiben Sie den vollständigen C-Code auf, um das Licht zyklisch im Uhrzeigersinn umlaufen zu lassen!

